

```

options(OutDec = ".") # Asegurar punto decimal

# Definición de variables para el problema
# Vector con posibles valores para los tiempos (en segundos)
# Tiempo de aumento de altura (25-35 seg)
tiempos_aumento <- as.integer(sample(seq(25, 35, by = 5), 1))
tiempos_pirqueta <- as.integer(sample(seq(3, 7, by = 1), 1))
tiempos_giro <- as.integer(sample(seq(15, 25, by = 5), 1))
tiempos_aterrizaje <- as.integer(sample(seq(20, 30, by = 5), 1))

# Alturas máximas posibles (en metros)
altura_maxima <- as.integer(sample(seq(100, 200, by = 20), 1))

# Definir variables para mostrar en el texto del problema
tiempo_total <- tiempos_aumento + tiempos_pirqueta + tiempos_giro + tiempos_aterrizaje

# Selección de gráficas
# Seleccionar aleatoriamente 3 de las 6 gráficas incorrectas
graficas_incorrectas_seleccionadas <- sample(1:6, 3)
# Seleccionar aleatoriamente 1 de las 3 gráficas correctas
grafica_correcta_seleccionada <- sample(1:3, 1)

# Determinar el índice de la respuesta correcta (entre 1 y 4)
indice_correcto <- sample(1:4, 1)

# Vector para controlar el orden de las gráficas
orden_graficas <- numeric(4)
orden_graficas[indice_correcto] <- grafica_correcta_seleccionada + 10 # Código para gráficas correctas
# Asignar las 3 gráficas incorrectas a las posiciones restantes
posiciones_incorrectas <- setdiff(1:4, indice_correcto)
for (i in 1:3) {
  orden_graficas[posiciones_incorrectas[i]] <- graficas_incorrectas_seleccionadas[i] # Códigos para in
}

# Vector para la solución
solucion <- numeric(4)
solucion[indice_correcto] <- 1

# Crear una variable para almacenar los chunks de Python
py_chunks <- character(4)

# Configuración general para las gráficas CON variables de R interpoladas
# Patrón correcto: definir variables Python al inicio del código
py_config <- paste0("
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np

# Configuración para eliminar etiquetas
plt.rcParams['figure.figsize'] = (8, 5)
plt.rcParams['figure.titlesize'] = 0

```

```

# Variables pasadas desde R (valores escalares)
t_aumento = ", tiempos_aumento, "
t_pirqueta = ", tiempos_pirqueta, "
t_giro = ", tiempos_giro, "
t_aterrizaje = ", tiempos_aterrizaje, "
h_max = ", altura_maxima, "
t_total = ", tiempo_total, "
")

# Colores para las gráficas correctas
colores_correctas <- c("'blue'", "'darkgreen'", "'purple'")
# Estilos de línea para gráficas correctas
estilos_linea <- c("'-'", "'--'", "'-.'")
# Grosos de línea para gráficas correctas
grosos_linea <- c(2, 2.5, 3)

# Generar el código Python para cada gráfica
for (i in 1:4) {
  tipo_grafica <- orden_graficas[i]

  # Gráficas correctas (11, 12, 13)
  if (tipo_grafica > 10) {
    num_correcta <- tipo_grafica - 10

    py_chunks[i] <- paste0(py_config, "
# Definir puntos clave para la gráfica correcta
# Fase 1: Aumento constante de altura
x1 = np.array([0, t_aumento])
y1 = np.array([0, h_max])

# Fase 2: Pirqueta (con pico correcto)
x2 = np.array([t_aumento, t_aumento + t_pirqueta/2, t_aumento + t_pirqueta])
y2 = np.array([h_max, h_max*1.5, h_max/2])

# Fase 3: Giro a altura constante
x3 = np.array([t_aumento + t_pirqueta, t_aumento + t_pirqueta + t_giro])
y3 = np.array([h_max/2, h_max/2])

# Fase 4: Aterrizaje (descenso constante)
x4 = np.array([t_aumento + t_pirqueta + t_giro, t_total])
y4 = np.array([h_max/2, 0])

# Crear figura
fig = plt.figure()
ax = fig.add_subplot(111)

# Color, estilo y grosor de línea específicos para esta variante correcta
color = ", colores_correctas[num_correcta], "
estilo = ", estilos_linea[num_correcta], "
grosor = ", grosos_linea[num_correcta], "

# Graficar cada segmento
ax.plot(x1, y1, color=color, linestyle=estilo, linewidth=grosor)

```

```

ax.plot(x2, y2, color=color, linestyle=estilo, linewidth=grosor)
ax.plot(x3, y3, color=color, linestyle=estilo, linewidth=grosor)
ax.plot(x4, y4, color=color, linestyle=estilo, linewidth=grosor)

# Configuración del gráfico
ax.set_xlabel('Segundos', fontweight='bold')
ax.set_ylabel('Altura (m)', fontweight='bold')
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_xlim(0, t_total + 5)
ax.set_ylim(0, h_max * 1.8)

# Mostrar gráfico
plt.subplots_adjust(top=1, bottom=0.1, left=0.1, right=0.9)
plt.savefig('grafica_', i, ".png", dpi=100)
plt.close()
")
}
# Gráficas incorrectas (1-6)
else {
    if (tipo_grafica == 1) {
        # Gráfica incorrecta 1: Mantener altura en vez de hacer pirueta
        py_chunks[i] <- paste0(py_config, "
# Gráfica incorrecta 1: Mantener altura en vez de hacer pirueta
# Fase 1: Aumento constante de altura
x1 = np.array([0, t_aumento])
y1 = np.array([0, h_max])

# Fase 2: Mantener altura en vez de hacer pirueta (incorrecto)
x2 = np.array([t_aumento, t_aumento + t_pirueta + t_giro])
y2 = np.array([h_max, h_max])

# Fase 3: Aterrizaje (descenso constante)
x3 = np.array([t_aumento + t_pirueta + t_giro, t_total])
y3 = np.array([h_max, 0])

# Crear figura
fig = plt.figure()
ax = fig.add_subplot(111)

# Graficar cada segmento
ax.plot(x1, y1, 'b-', linewidth=2)
ax.plot(x2, y2, 'b-', linewidth=2)
ax.plot(x3, y3, 'b-', linewidth=2)

# Configuración del gráfico
ax.set_xlabel('Segundos', fontweight='bold')
ax.set_ylabel('Altura (m)', fontweight='bold')
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_xlim(0, t_total + 5)
ax.set_ylim(0, h_max * 1.8)

# Mostrar gráfico
plt.subplots_adjust(top=1, bottom=0.1, left=0.1, right=0.9)

```

```

plt.savefig('grafica_', i, ".png", dpi=100)
plt.close()
")
}
else if (tipo_grafica == 2) {
  # Gráfica incorrecta 2: Solo muestra descenso en la pirueta sin el pico
  py_chunks[i] <- paste0(py_config, "
# Gráfica incorrecta 2: Solo muestra descenso en la pirueta sin el pico
# Fase 1: Aumento constante de altura
x1 = np.array([0, t_aumento])
y1 = np.array([0, h_max])

# Fase 2: Pirueta (solo descenso constante, incorrecto)
x2 = np.array([t_aumento, t_aumento + t_pirueta])
y2 = np.array([h_max, h_max/2])

# Fase 3: Giro a altura constante
x3 = np.array([t_aumento + t_pirueta, t_aumento + t_pirueta + t_giro])
y3 = np.array([h_max/2, h_max/2])

# Fase 4: Aterrizaje (descenso constante)
x4 = np.array([t_aumento + t_pirueta + t_giro, t_total])
y4 = np.array([h_max/2, 0])

# Crear figura
fig = plt.figure()
ax = fig.add_subplot(111)

# Graficar cada segmento
ax.plot(x1, y1, 'g-', linewidth=2)
ax.plot(x2, y2, 'g-', linewidth=2)
ax.plot(x3, y3, 'g-', linewidth=2)
ax.plot(x4, y4, 'g-', linewidth=2)

# Configuración del gráfico
ax.set_xlabel('Segundos', fontweight='bold')
ax.set_ylabel('Altura (m)', fontweight='bold')
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_xlim(0, t_total + 5)
ax.set_ylim(0, h_max * 1.8)

# Mostrar gráfico
plt.subplots_adjust(top=1, bottom=0.1, left=0.1, right=0.9)
plt.savefig('grafica_', i, ".png", dpi=100)
plt.close()
")
}
else if (tipo_grafica == 3) {
  # Gráfica incorrecta 3: Con una fase de vuelo adicional
  py_chunks[i] <- paste0(py_config, "
# Gráfica incorrecta 3: Con una fase de vuelo adicional
# Fase 1: Aumento constante de altura
x1 = np.array([0, t_aumento])

```

```

y1 = np.array([0, h_max])

# Fase 2: Pirueta (descenso constante)
x2 = np.array([t_aumento, t_aumento + t_pirueta])
y2 = np.array([h_max, h_max/3])

# Fase EXTRA: Elevación adicional (incorrecto)
x_extra = np.array([t_aumento + t_pirueta, t_aumento + t_pirueta + 5])
y_extra = np.array([h_max/3, h_max])

# Fase 3: Giro a altura constante
x3 = np.array([t_aumento + t_pirueta + 5, t_aumento + t_pirueta + 5 + t_giro])
y3 = np.array([h_max, h_max])

# Fase 4: Aterrizaje (descenso constante)
x4 = np.array([t_aumento + t_pirueta + 5 + t_giro, t_aumento + t_pirueta + 5 + t_giro + t_aterrizaje])
y4 = np.array([h_max, 0])

# Crear figura
fig = plt.figure()
ax = fig.add_subplot(111)

# Graficar cada segmento
ax.plot(x1, y1, color='red', linestyle='--', linewidth=2)
ax.plot(x2, y2, color='red', linestyle='--', linewidth=2)
ax.plot(x_extra, y_extra, color='red', linestyle='--', linewidth=2)
ax.plot(x3, y3, color='red', linestyle='--', linewidth=2)
ax.plot(x4, y4, color='red', linestyle='--', linewidth=2)

# Configuración del gráfico
ax.set_xlabel('Segundos', fontweight='bold')
ax.set_ylabel('Altura (m)', fontweight='bold')
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_xlim(0, t_aumento + t_pirueta + 5 + t_giro + t_aterrizaje + 5)
ax.set_ylim(0, h_max * 1.8)

# Mostrar gráfico
plt.subplots_adjust(top=1, bottom=0.1, left=0.1, right=0.9)
plt.savefig('grafica_', i, ".png", dpi=100)
plt.close()
")
}
else if (tipo_grafica == 4) {
    # Gráfica incorrecta 4: Pirueta con pico pero sin descenso
    py_chunks[i] <- paste0(py_config, "
# Gráfica incorrecta 4: Pirueta con pico pero sin descenso
# Fase 1: Aumento constante de altura
x1 = np.array([0, t_aumento])
y1 = np.array([0, h_max])

# Fase 2: Pirueta (con pico pero sin descenso, incorrecto)
x2 = np.array([t_aumento, t_aumento + t_pirueta/2, t_aumento + t_pirueta])
y2 = np.array([h_max, h_max*1.5, h_max])

```

```

# Fase 3: Giro a altura constante
x3 = np.array([t_aumento + t_pirueta, t_aumento + t_pirueta + t_giro])
y3 = np.array([h_max, h_max])

# Fase 4: Aterrizaje (descenso constante)
x4 = np.array([t_aumento + t_pirueta + t_giro, t_total])
y4 = np.array([h_max, 0])

# Crear figura
fig = plt.figure()
ax = fig.add_subplot(111)

# Graficar cada segmento
ax.plot(x1, y1, 'C1', linewidth=2)
ax.plot(x2, y2, 'C1', linewidth=2)
ax.plot(x3, y3, 'C1', linewidth=2)
ax.plot(x4, y4, 'C1', linewidth=2)

# Configuración del gráfico
ax.set_xlabel('Segundos', fontweight='bold')
ax.set_ylabel('Altura (m)', fontweight='bold')
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_xlim(0, t_total + 5)
ax.set_ylim(0, h_max * 1.8)

# Mostrar gráfico
plt.subplots_adjust(top=1, bottom=0.1, left=0.1, right=0.9)
plt.savefig('grafica_', i, ".png", dpi=100)
plt.close()
")
}
else if (tipo_grafica == 5) {
    # Gráfica incorrecta 5: Oscilaciones durante el giro
    py_chunks[i] <- paste0(py_config, "
# Gráfica incorrecta 5: Oscilaciones durante el giro
# Fase 1: Aumento constante de altura
x1 = np.array([0, t_aumento])
y1 = np.array([0, h_max])

# Fase 2: Pirueta (con pico correcto)
x2 = np.array([t_aumento, t_aumento + t_pirueta/2, t_aumento + t_pirueta])
y2 = np.array([h_max, h_max*1.5, h_max/2])

# Fase 3: Giro con oscilaciones (incorrecto)
t = np.linspace(t_aumento + t_pirueta, t_aumento + t_pirueta + t_giro, 100)
oscilacion = (h_max/2) * 0.2 * np.sin(10 * (t - (t_aumento + t_pirueta)))
altura_base = np.ones_like(t) * (h_max/2)
x3 = t
y3 = altura_base + oscilacion

# Fase 4: Aterrizaje (descenso constante)
x4 = np.array([t_aumento + t_pirueta + t_giro, t_total])
y4 = np.array([h_max/2, 0])

```

```

# Crear figura
fig = plt.figure()
ax = fig.add_subplot(111)

# Graficar cada segmento
ax.plot(x1, y1, color='purple', linestyle='-', linewidth=2)
ax.plot(x2, y2, color='purple', linestyle='-', linewidth=2)
ax.plot(x3, y3, color='purple', linestyle='-', linewidth=2)
ax.plot(x4, y4, color='purple', linestyle='-', linewidth=2)

# Configuración del gráfico
ax.set_xlabel('Segundos', fontweight='bold')
ax.set_ylabel('Altura (m)', fontweight='bold')
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_xlim(0, t_total + 5)
ax.set_ylim(0, h_max * 1.8)

# Mostrar gráfico
plt.subplots_adjust(top=1, bottom=0.1, left=0.1, right=0.9)
plt.savefig('grafica_', i, ".png", dpi=100)
plt.close()
")
}
else if (tipo_grafica == 6) {
    # Gráfica incorrecta 6: Tiempos incorrectos en las fases
    py_chunks[i] <- paste0(py_config, "
# Gráfica incorrecta 6: Tiempos incorrectos en las fases
# Calcular tiempos incorrectos dentro de Python
tiempo_incorrecto1 = t_aumento / 2
pirueta_doble = t_pirueta * 2
fin_fase2 = tiempo_incorrecto1 + pirueta_doble
fin_fase3 = fin_fase2 + t_giro + (t_aumento - tiempo_incorrecto1)

# Fase 1: Aumento constante de altura (tiempo incorrecto)
x1 = np.array([0, tiempo_incorrecto1])
y1 = np.array([0, h_max])

# Fase 2: Pirueta (con pico correcto pero tiempo incorrecto)
x2 = np.array([tiempo_incorrecto1, tiempo_incorrecto1 + t_pirueta, fin_fase2])
y2 = np.array([h_max, h_max*1.5, h_max/2])

# Fase 3: Giro a altura constante (tiempo más largo para compensar)
x3 = np.array([fin_fase2, fin_fase3])
y3 = np.array([h_max/2, h_max/2])

# Fase 4: Aterrizaje (descenso constante)
x4 = np.array([fin_fase3, t_total])
y4 = np.array([h_max/2, 0])

# Crear figura
fig = plt.figure()
ax = fig.add_subplot(111)

```

```

# Graficar cada segmento
ax.plot(x1, y1, color='brown', linestyle='-', linewidth=2)
ax.plot(x2, y2, color='brown', linestyle='-', linewidth=2)
ax.plot(x3, y3, color='brown', linestyle='-', linewidth=2)
ax.plot(x4, y4, color='brown', linestyle='-', linewidth=2)

# Configuración del gráfico
ax.set_xlabel('Segundos', fontweight='bold')
ax.set_ylabel('Altura (m)', fontweight='bold')
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_xlim(0, t_total + 5)
ax.set_ylim(0, h_max * 1.8)

# Mostrar gráfico
plt.subplots_adjust(top=1, bottom=0.1, left=0.1, right=0.9)
plt.savefig('grafica_', i, ".png", dpi=100)
plt.close()
")
    }
}
}

# Reiniciar sesión de Python para evitar conflictos con variables anteriores
reticulate::py_run_string("import importlib; import sys")

# Ejecutar todos los chunks de Python para generar las gráficas
for (i in 1:4) {
  tryCatch({
    py_run_string(py_chunks[i])
  }, error = function(e) {
    cat("Error en gráfica", i, ":", conditionMessage(e), "\n")
    cat("Código generado:\n", substr(py_chunks[i], 1, 500), "\n")
  })
}

```

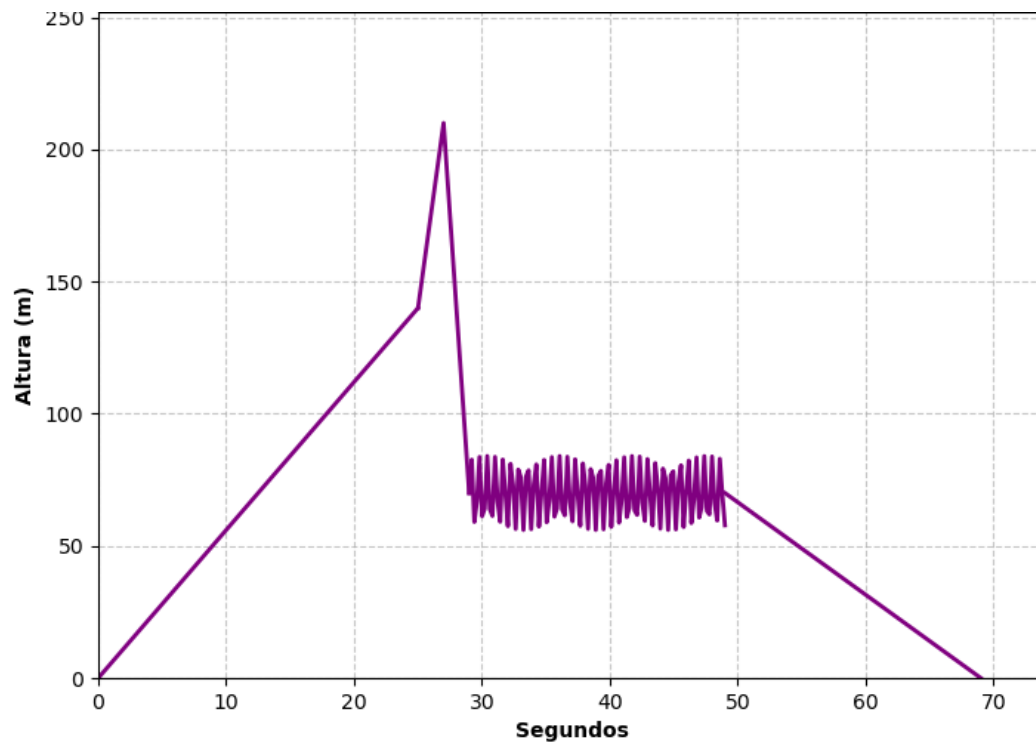
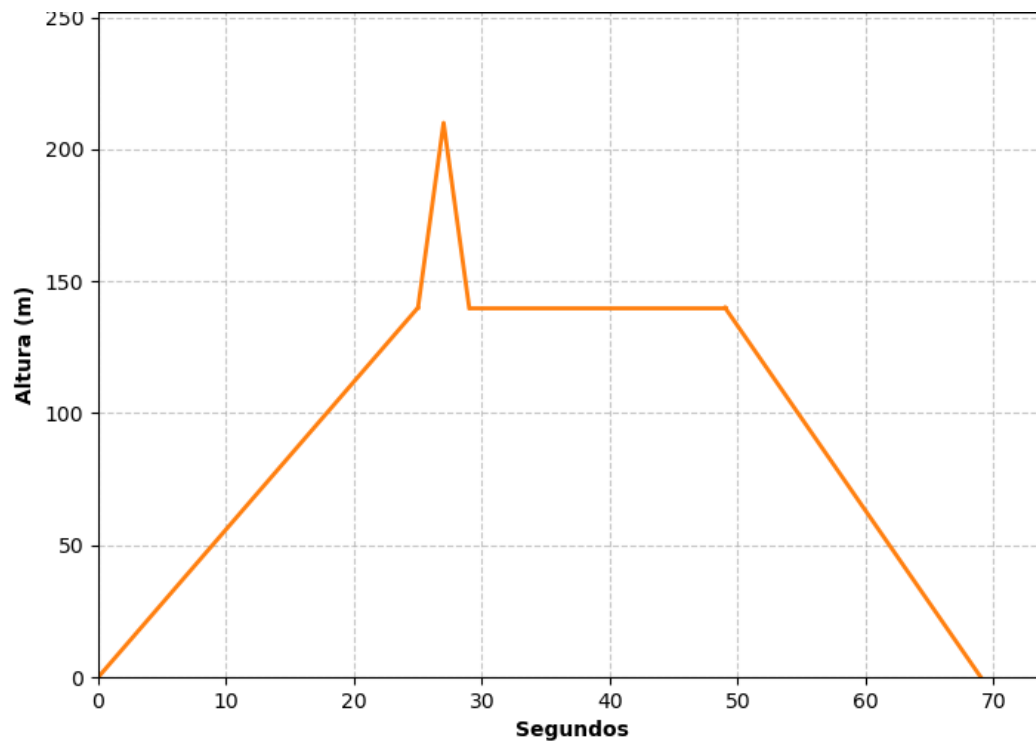
Question

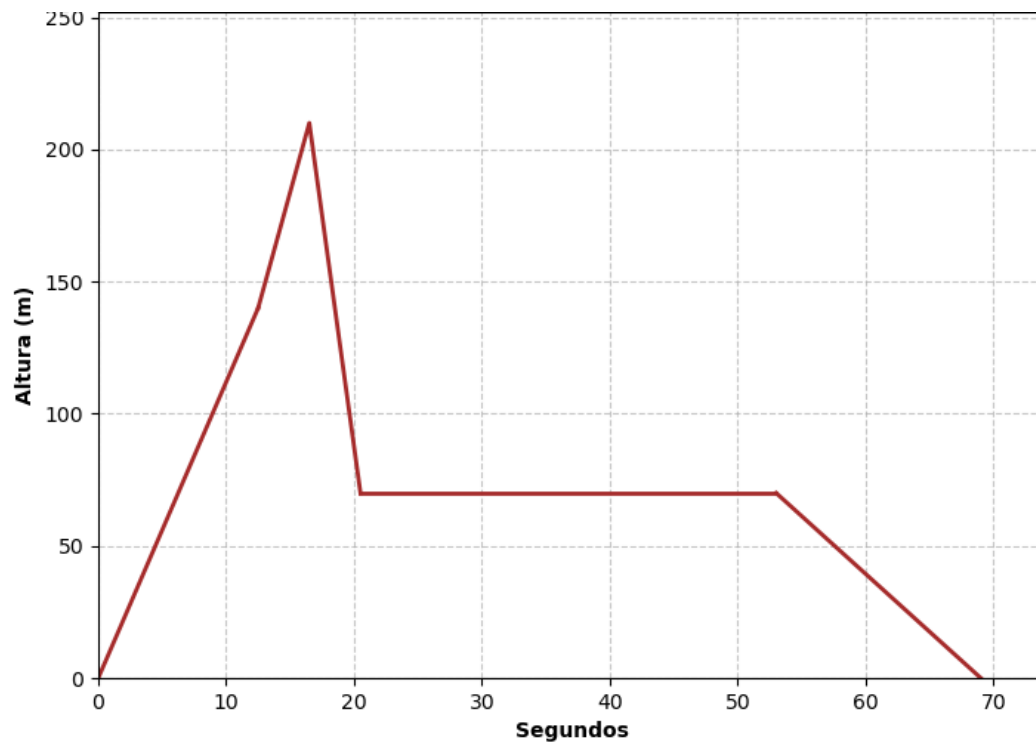
En una entrevista para una cadena de televisión, un piloto de un avión de acrobacias comenta en detalle cómo fue su presentación:

“Durante los primeros 25 segundos aumenté la altura de manera constante y luego, realicé una pirueta espectacular donde primero elevé el avión a una altura aún mayor y después descendí bruscamente. Esto duró 4 segundos, quedando a la mitad de la altura máxima inicial. Después de la pirueta, comencé a girar en torno al eje del avión durante 20 segundos, manteniendo la misma altura. Para finalizar, aterricé reduciendo la altura de manera constante durante 20 segundos”.

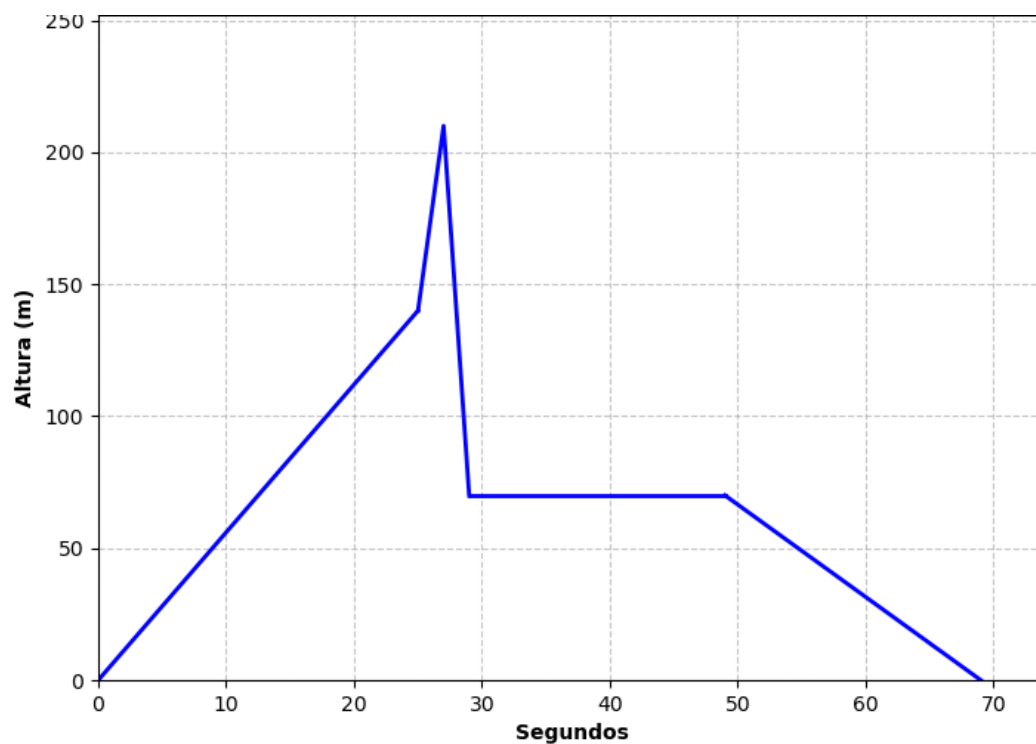
¿Cuál de las siguientes gráficas representa de forma CORRECTA la información dada por el piloto?

Answerlist





•



•

Solution

La gráfica correcta es la que representa adecuadamente las cuatro fases del vuelo acrobático descritas por el piloto:

1. Durante los primeros 25 segundos: aumento constante de altura (línea ascendente).
2. Pirueta espectacular donde primero elevó el avión a una altura mayor y después descendió bruscamente durante 4 segundos, quedando a la mitad de la altura máxima inicial (pico hacia arriba y luego descenso).
3. Giro alrededor del eje del avión durante 20 segundos manteniendo la misma altura (línea horizontal).
4. Aterrizaje con reducción constante de altura durante 20 segundos (línea descendente final).

Las demás opciones contienen elementos incorrectos que no corresponden con la descripción dada por el piloto. Estos errores pueden incluir:

- Mantener la altura constante después del ascenso inicial, sin mostrar la pirueta con el pico hacia arriba y el descenso.
- Mostrar solo un descenso constante durante la pirueta, sin el pico hacia arriba que el piloto describió.
- Incluir una fase adicional de elevación después de la pirueta, la cual no fue mencionada por el piloto.
- Mostrar la pirueta con un pico hacia arriba pero sin el descenso posterior, volviendo a la altura inicial.
- Mostrar oscilaciones durante la fase de giro, cuando el piloto especificó que mantuvo la misma altura.
- Presentar tiempos incorrectos en las diferentes fases del vuelo.

Meta-information

exname: vuelo_acrobatico(single-choice) exctype: schoice exsolution: 0001 exshuffle: TRUE exsection: Interpretación de gráficas